Contents lists available at ScienceDirect
journal homepage: www.elsevier.com/locate/softx

FedGuard-IIoT: A reproducible software pipeline for communication-aware and Byzantine-robust federated anomaly detection in industrial IoT

Sundetkhan Bekzat, Baibolat Bekarys

Course project manuscript prepared for reproducible software submission practice. Institutional affiliation and corresponding author details require author confirmation.

HIGHLIGHTS

- *Open-source pipeline for federated IIoT anomaly detection*
- *Integrates LSTM-AE, FedProx, Top-k, and robust aggregation*
- *Reproduces benchmark runs, JSON outputs, and article figures*
- *Exposes communication-quality trade-offs under client heterogeneity*
- *Compares aggregation behavior under Byzantine update poisoning*

ARTICLE INFO

Keywords: *federated learning; anomaly detection; industrial IoT; Byzantine robustness; reproducible software; PyTorch*

ABSTRACT

FedGuard-IIoT is a reproducible software pipeline for federated anomaly detection in industrial IoT settings with three practical constraints: limited uplink bandwidth, heterogeneous client data, and potentially poisoned model updates. The repository combines four elements in one executable workflow: synthetic multi-client data generation, local LSTM autoencoder training, Top-k compression with error feedback, FedProx-based stabilization, and configurable robust aggregation on the server. The implementation is written in Python and PyTorch, stores experiment outputs in JSON, and regenerates the paper figures from code. In the current five-client synthetic benchmark, the 10% compression setting reduces communication from 14.92 MB to 1.49 MB while keeping AUROC close to full transmission, and robust aggregators degrade less sharply than FedAvg as the malicious-client fraction increases. The novelty of the work is not a new federated optimization rule; it is the integration of communication reduction, heterogeneity handling, and Byzantine-robust aggregation into one reproducible experimental pipeline that can be rerun, inspected, and extended.

Nr.	Code metadata description	Metadata
(C1)	Current code version	0.1.0 (from `pyproject.toml`)
(C2)	Permanent link to code/repository used for this code version	`https://github.com/Bekzat158/multi-agent-system-assig`
(C3)	Permanent link to reproducible capsule	Requires author completion. No archived release, DOI, or reproducible capsule was found in the repository.
(C4)	Legal code license	Requires author completion. No top-level `LICENSE` file was found in the repository.
(C5)	Code versioning system used	Git
(C6)	Software code languages, tools, and services used	Python 3.12, PyTorch, NumPy, scikit-learn, matplotlib, seaborn, SciPy, pandas, `uv`, Typst
(C7)	Compilation requirements, operating environments & dependencies	Python >= 3.12; dependencies declared in `pyproject.toml`; manuscript builds with `typst compile`; experiments select CUDA when available and otherwise run on CPU.
(C8)	If available Link to developer documentation/manual	`README.md` in the repository. External documentation link requires author completion if available.
(C9)	Support email for questions	Requires author completion.

***Table 1**: Code metadata.*

***Note:** archival release, software license, affiliation details, and a support contact should be completed before submission.*

1 Motivation and significance

Federated anomaly detection is relevant to industrial Internet of Things deployments because raw sensor streams are often costly or impractical to centralize continuously. In practice, however, a usable pipeline must handle three constraints at once: limited communication budgets, heterogeneous operating regimes across clients, and the risk of poisoned updates during training. These topics are well covered individually in the literature, but they are less often exposed together in a small, reproducible software artifact.

FedGuard-IIoT addresses this gap as a software integration problem. The repository does not claim a new anomaly score, a new compression rule, or a new robust estimator. Its contribution is to combine an LSTM autoencoder, Top-k error-feedback compression, FedProx stabilization, and robust server aggregation in one round-based workflow with stored configurations, generated figures, and serialized results. This makes it possible

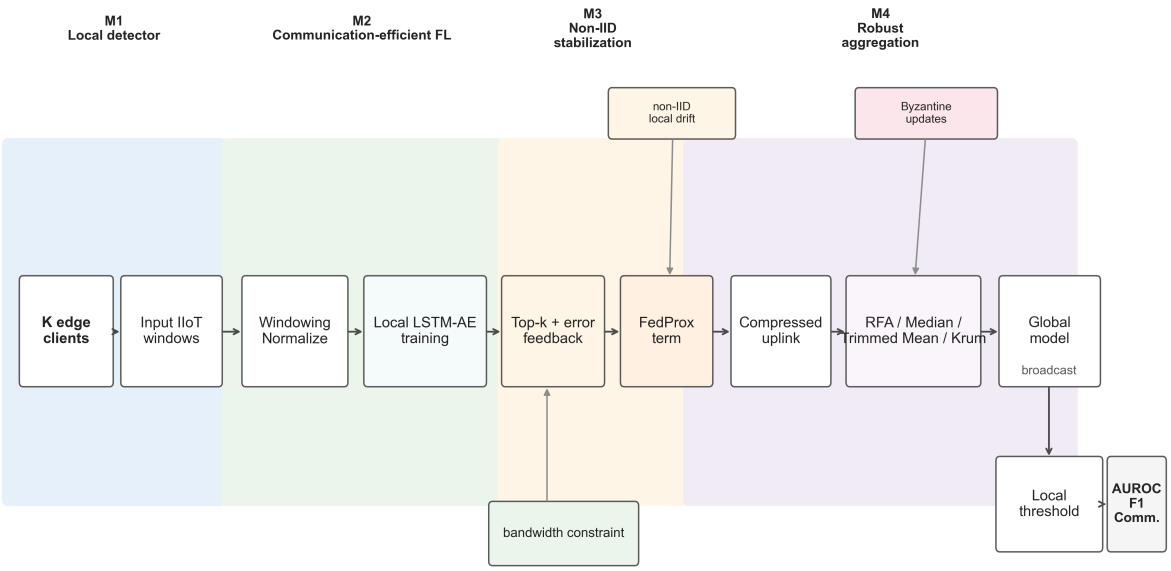

***Figure 1**: Integrated workflow of FedGuard-IIoT. Client windows are used for local LSTM autoencoder training, optional FedProx regularization and Top-k error-feedback compression are applied before upload, and the server aggregates updates with a selected rule before broadcasting the revised global model.*

to study how these components interact under one shared benchmark instead of comparing them across separate codebases.

The software serves two practical purposes. It can be used as a compact reference implementation for reproducible experiments on communication-aware and adversarial federated anomaly detection, and it can be used as a testbed for changing one module at a time while keeping the rest of the workflow fixed.

The repository also addresses a reproducibility gap. The benchmark data are generated procedurally from fixed seeds, the main script reruns the full workflow from data creation to figure export, and the results are written to a machine-readable JSON file. The benchmark is still synthetic and modest in scale, but the full pipeline can be rerun without reconstructing the experiments from prose alone.

2 Software description

2.1 Overview

The repository is organized around one executable workflow in `run_experiments.py`*. A typical run proceeds through dataset generation, baseline local pretraining, federated experiments, training of a representative final model for visualization, result serialization, and figure generation. The main source files are* `src/data_gen.py` *for synthetic data construction,* `src/models.py` *for anomaly detection models and scoring utilities,* `src/fl_engine.py` *for client training and aggregation,* `src/attacks.py` *for adversarial update manipulation,* `src/experiments.py` *for experiment orchestration, and* `src/visualization.py` *for figure generation.*

The core workflow is shown in Figure 1. Each client receives its own multivariate time-series windows, performs local training on an LSTM autoencoder, and returns a model update rather than raw data. If compression is enabled, the update is sparsified with Top-k selection and corrected with error feedback. The server aggregates client updates with either vanilla averaging or a robust rule, updates the global model, and broadcasts new parameters for the next round. Evaluation is based on reconstruction scores and stores summary metrics together with communication traces.

2.2 Core modules and integrated workflow

Module	***Purpose***	***Implementation in repository***	***Observable output***
Local anomaly detector	*Learn reconstruction-based anomaly scores from client windows*	`src/models.py`*: single-layer LSTM autoencoder; score from per-window reconstruction error*	*Scores used for AUROC, F1, and example anomaly plots*
Communication-efficient federated training	*Reduce uplink cost during training*	`src/fl_engine.py`*: Top-k sparsification with per-client residuals and communication tracking*	*Per-round and cumulative transmitted bytes*
Non-IID stabilization	*Reduce drift between local and global objectives*	`src/fl_engine.py`*: FedProx local objective with configurable* `mu`	*Smoother training trajectories in convergence studies*
Byzantine-robust aggregation	*Reduce sensitivity to corrupted updates*	`src/fl_engine.py`*: coordinate-wise median, trimmed mean, Krum, and RFA*	*Robustness curves under increasing attack fractions*

***Table 2**: Core software modules in the integrated workflow.*

The local model is intentionally compact. The stored experiment configuration uses a six-channel input, window size 30, hidden size 48, latent size 12, and one LSTM layer. This compactness matters because the software is intended to make communication and aggregation effects visible without hiding them behind a very large model.

The non-IID and robustness modules are implemented as part of the same round logic rather than as separate scripts. `federated_training` *selects participating clients, runs local training with either FedAvg or FedProx, applies attack logic*

where requested, optionally compresses updates, aggregates them on the server, and records round-level communication. This integrated execution path is the main software contribution, because it makes the interaction between the four components observable in one place.

The repository also includes additional code that is not central to the reported SoftwareX workflow, such as a lightweight `DeepSVDD` *class and an* `FLTrust`*-style aggregator helper. These are not used in the reported experiments and are therefore not claimed as validated components of the present article.*

2.3 Software architecture and implementation

The data generator constructs a five-client synthetic benchmark in which clients differ by base frequency, amplitude, and noise level. Each client contributes 4000 samples over six channels. Normal signals are built from sinusoidal components and harmonics with additive noise and per-feature offsets. Three anomaly types are injected: spikes, drift segments, and pattern shifts. The generator writes train, test, and label arrays to `data/` *and uses fixed seeds to make the benchmark deterministic across reruns.*

The training engine is implemented in plain PyTorch. `local_train_fedavg` *and* `local_train_fedprox` *share the same reconstruction objective, while FedProx adds a proximal term with coefficient* `mu` *to restrain drift from the broadcast model. The compression layer is implemented through* `TopKCompressor`*, which keeps the largest-magnitude coordinates and stores the removed residual per client for reinjection in later rounds. Communication is tracked analytically from the number of active parameters and the configured bit width.*

The server-side aggregation layer exposes both baseline and robust rules. FedAvg uses weighted averaging by local sample count. Coordinate-wise median and trimmed mean operate per parameter coordinate, Krum performs distance-based neighbor selection, and robust federated aggregation approximates the geometric median through Weiszfeld-style iterations. This makes the repository useful as a comparative testbed rather than a single fixed method implementation.

Evaluation is also implemented in code rather than described abstractly. `src/experiments.py` *computes AUROC, F1, and AUPR from reconstruction scores and writes the outputs to* `results/experiment_results.json`*. The current evaluation script reports pooled metrics using a global 90th-percentile threshold over aggregated scores for comparability across configurations. The software also includes threshold utilities that support percentile-based thresholding at the client level for local inspection and visualization. The article reports the implementation this way to match the code as stored.*

2.4 Dependencies, operating environment, and reproducibility notes

The project declares its dependencies in `pyproject.toml` *and uses* `uv` *for environment management. The main runtime dependencies are PyTorch, NumPy, scikit-learn, matplotlib, seaborn, pandas, SciPy, and tqdm. The article source is written in Typst and can be compiled with the bundled* `typst` *binary.*

The main entry point for reproducing the paper is:

```
uv sync
uv run python run_experiments.py
typst compile sn-article.typ sn-article.pdf
```

The experiment script automatically selects CUDA when available and otherwise falls back to CPU. Fixed seeds are used in dataset generation and federated client sampling. The output artifacts are stored in `data/`*,* `results/`*, and* `figures/`*. Because the benchmark is synthetic, there is no external data acquisition step.*

Some SoftwareX reproducibility requirements are not yet fully satisfied by the present repository state. No top-level software license was found, no archived release or DOI was found, and no external documentation site beyond `README.md` *was identified. These are repository-completion tasks rather than manuscript claims, and they should be addressed before submission.*

3 Illustrative examples

3.1 Example workflow execution

The first illustrative use of the software is the end-to-end federated experiment workflow itself. Running `run_experiments.py` *generates the five-client synthetic dataset, trains the baseline local model, executes convergence, communication, robustness, ablation, and on-off attack studies, stores structured outputs in JSON, and redraws the figure set. The result is a single reproducible run path rather than a set of disconnected scripts or notebooks.*

The communication study shows how the compression module is meant to be used. In the dedicated Top-k sweep, reducing the transmitted fraction from 100% to 10% lowers the communication volume from 14.92 MB to 1.49 MB while AUROC changes from 0.663 to 0.658. At 1%, communication falls further to 0.149 MB but AUROC decreases to 0.632. In this benchmark, 10% is therefore a reasonable operating point, while more aggressive compression comes with a clearer accuracy cost.

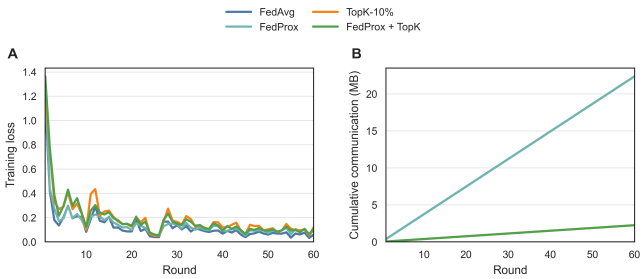

***Figure 2**: Illustrative convergence and communication traces from the federated training engine. FedProx smooths the training curve under heterogeneous clients, while Top-k compression substantially reduces cumulative communication.*

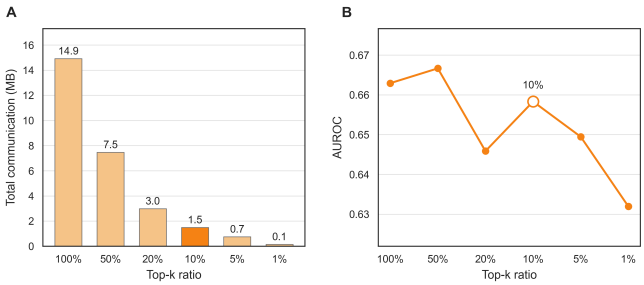

***Figure 3**: Communication-quality trade-off across Top-k compression ratios. The 10% setting provides a practical operating point in the current benchmark, retaining similar AUROC to full transmission at much lower communication volume.*

3.2 Example outputs and behavior under key settings

The anomaly-detection output in Figure 4 illustrates the local inference stage after federated training. On a representative client, reconstruction error rises around injected anomalous

intervals and stays lower during normal regions. This figure connects the round-level training results to the actual detection output produced by the pipeline.

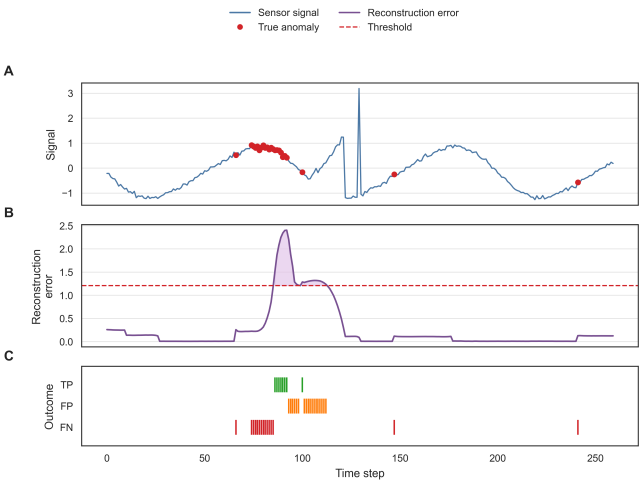

***Figure 4**: Representative local anomaly-detection output generated by the software after federated training. The plot combines sensor signal, reconstruction score, threshold, and detection outcomes for one client.*

The robustness study shows why the aggregation layer is exposed as a configurable module. Under increasing fractions of malicious clients, FedAvg degrades sharply, whereas median and RFA remain more stable in the reported runs. The best rule is not identical across all conditions, which is why the repository keeps several aggregation options available under the same workload.

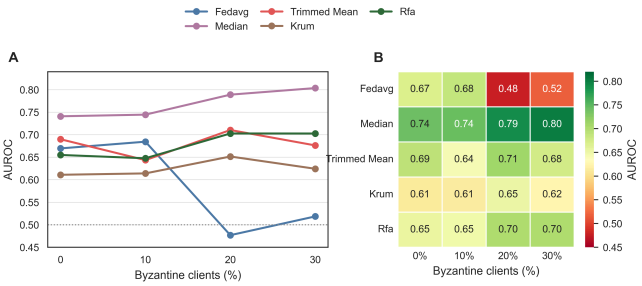

***Figure 5**: Illustrative robustness output from the aggregation module. The software reports AUROC across attack fractions for FedAvg, median, trimmed mean, Krum, and RFA, making it possible to compare defensive behavior under the same workload.*

The ablation study shows that the four modules contribute differently. In the stored results, removing robust aggregation causes the largest performance drop under attack, while removing compression mainly increases communication cost. The full configuration is therefore best read as a balanced default workflow for this benchmark rather than as a uniformly best setting on every metric.

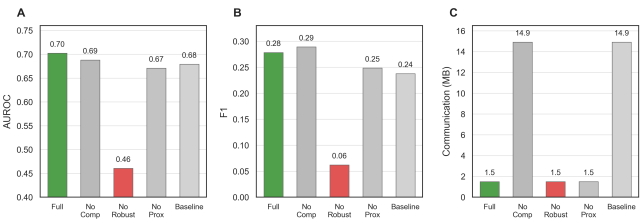

***Figure 6**: Ablation output from the integrated pipeline. The figure illustrates how communication, stabilization, and robustness modules contribute differently when the full workflow is stressed by adversarial participation.*

4 Impact

The main impact of FedGuard-IIoT is reuse through software. The repository provides a compact platform for studying how communication compression, non-IID stabilization, and robust aggregation behave together in federated anomaly detection. This comparison is harder when each component is packaged in a separate codebase, evaluated on a different workload, or reported without reproducible result generation.

The software is also suitable for teaching and benchmarking. Its source layout is small enough to inspect in full, yet broad enough to demonstrate a federated workflow with attacks, communication accounting, and figure reproduction. Because outputs are stored in JSON and the figures are regenerated from code, the project can be extended by replacing the local detector, introducing alternative client-selection policies, adding new robust aggregators, or swapping the synthetic benchmark for a real industrial dataset.

4.1 Limitations and repository readiness

The reported benchmark is synthetic. It captures heterogeneous periodicities, amplitudes, noise levels, and anomaly types, but it does not reproduce plant-level control loops, maintenance events, or operator interventions. The software should therefore be read as a reproducible reference pipeline, not as a deployment-ready industrial product.

The current repository also has packaging limitations relevant to SoftwareX submission. The absence of a visible license prevents clear reuse conditions. No archived release or DOI was found, which weakens permanent reproducibility. The README is serviceable as a starting manual, but a more explicit user guide and release workflow would improve reuse.

At the method level, strong compression eventually degrades performance, a single shared encoder may be insufficient for strongly personalized clients, and adaptive adversaries could erode the present robust aggregation rules. These are limitations of the current software configuration, not claims of universal failure.

5 Conclusions and future work

FedGuard-IIoT presents communication-aware and adversarial federated anomaly detection as a reproducible software artifact. The repository integrates synthetic data generation, local LSTM autoencoder training, Top-k error-feedback compression, FedProx stabilization, robust aggregation, JSON result storage, and figure regeneration in one executable workflow. The reported runs show how the current implementation exposes trade-offs between communication cost, anomaly-detection quality, and robustness under attack.

The novelty of the work is software-centered rather than algorithmic. FedGuard-IIoT brings together compression, heterogeneity handling, and Byzantine-robust aggregation in one reproducible pipeline that can be rerun, inspected, and extended. Future work should focus on completing the repository for archival submission, adding real industrial datasets, separating client-level and pooled evaluation more explicitly, and packaging the code with a permanent release and clear license.

CRediT authorship contribution statement

Sundetkhan Bekzat: Conceptualization, Software, Investigation, Visualization, Writing - original draft. Baibolat Bekarys: Soft-

ware, Validation, Writing - review & editing. Author confirmation required.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this article.

Acknowledgements

The authors thank Seema Rawat for course supervision and feedback during the development of the project from which this software paper was derived. Author confirmation required.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors. Author confirmation required.

Data availability

No external dataset was used. The experiments rely on a procedurally generated synthetic benchmark created by the repository from fixed random seeds and stored locally when `run_experiments.py` *is executed. The source code and generated results are available in the project repository at* `https://github.com/Bekzat158/multi-agent-system-assig`*. A permanent archived release and any related DOI require author completion.*

References

[1] *McMahan, B., Moore, E., Ramage, D., Hampson, S., & Agüera y Arcas, B. (2017). Communication-efficient learning of deep networks from decentralized data.* AISTATS 2017*, PMLR 54, 1273–1282.*

[2] *Li, T., Sahu, A. K., Zaheer, M., Sanjabi, M., Talwalkar, A., & Smith, V. (2020). Federated optimization in heterogeneous networks.* MLSys 2020.

[3] *Reisizadeh, A., Mokhtari, A., Hassani, H., Jadbabaie, A., & Pedarsani, R. (2020). FedPAQ: A communication-efficient federated learning method with periodic averaging and quantization.* AISTATS 2020*, PMLR 108.*

[4] *Blanchard, P., Mhamdi, E. M. E., Guerraoui, R., & Stainer, J. (2017). Machine learning with adversaries: Byzantine tolerant gradient descent.* NeurIPS 2017.

[5] *Yin, D., Chen, Y., Kannan, R., & Bartlett, P. (2018). Byzantine-robust distributed learning: Towards optimal statistical rates.* ICML 2018*, PMLR 80.*

[6] *Pillutla, K., Kakade, S. M., & Harchaoui, Z. (2022). Robust aggregation for federated learning.* IEEE Transactions on Signal Processing*, 70, 1142–1154.*

[7] *Cao, X., Fang, M., Liu, J., & Gong, N. Z. (2021). FLTrust: Byzantine-robust federated learning via trust bootstrapping.* NDSS 2021.

[8] *Bagdasaryan, E., Veit, A., Hua, Y., Estrin, D., & Shmatikov, V. (2020). How to backdoor federated learning.* AISTATS 2020*, PMLR 108.*

[9] *Fang, M., Cao, X., Jia, J., & Gong, N. Z. (2020). Local model poisoning attacks to Byzantine-robust federated learning.* USENIX Security 2020.

[10] *Xie, C., Chen, M., Chen, P.-Y., & Li, B. (2023). Attacks against federated learning defense systems and their implications.* JMLR*, 24(305), 1–43.*

[11] *Yuan, D., Hu, S., Guo, S., Zhang, J., & Yang, B. (2020). Deep anomaly detection for time-series data in industrial IoT: A communication-efficient on-device federated learning approach.* IEEE Internet of Things Journal*, 8(9), 6348–6358.*

[12] *Audibert, J., Michiardi, P., Guyard, F., Marti, S., & Zuluaga, M. A. (2022). Deep learning for anomaly detection in time series: A survey.* ACM Computing Surveys*, 54(3).*

[13] *Park, D., Hoshi, Y., & Kemp, C. C. (2018). A multimodal anomaly detector for robot-assisted feeding using an LSTM-based variational autoencoder.* IEEE Robotics and Automation Letters*, 4(2), 1543–1550.*